

02분반

소프트웨어캡스톤디자인

에스파팀 결과보고서

김진혁, 박홍주, 송태웅, 이동민, 이연제

TABLE OF CONTENTS

목차 소개

01 | 팀 소개

02 | 프로젝트 소개

03 | 시스템 설계
• 시스템 설계
• 기술 스택

04 | AI
• DDPM
• MolScribe
• AI 학습 전략

05 | 백엔드
• 서버 설계
• Spring boot
• ERD

06 | 프론트 (안드로이드)
• 3D 랜더링
• 스토리보드

01

팀 소개

“

A.E.S.F.A(Android Evolve Spring Fastapi AI)

”

에스파



AI

20193317 빅데이터 김진혁 (팀장)
20195189 빅데이터 송태웅

Backend

20213242 콘텐츠IT 이동민
20195189 빅데이터 송태웅

Frontend

20193236 데이터테크 이연제
20205181 콘텐츠IT 박홍주
20195189 빅데이터 송태웅

02

프로젝트 소개

프로젝트 명: CanChem

- 분자 정보 제공 앱
- AI를 활용한 분자구조 이미지 인식



CanChem 시스템 설계



03

기술 스택



백엔드 기술 스택

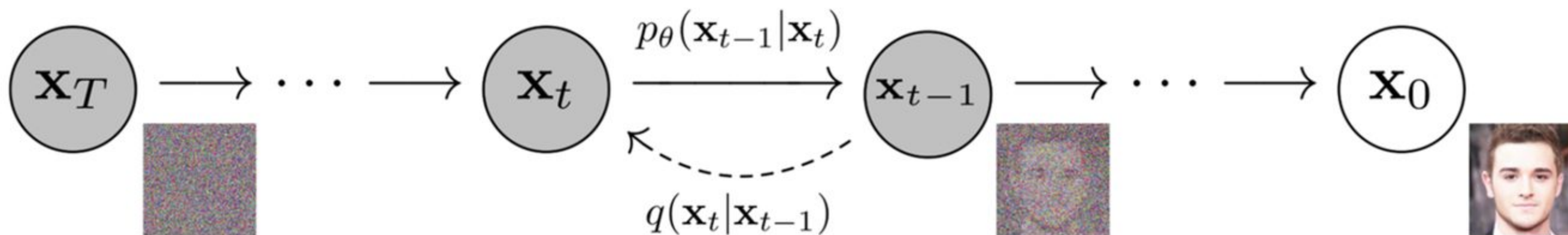


» AI 기술 스택

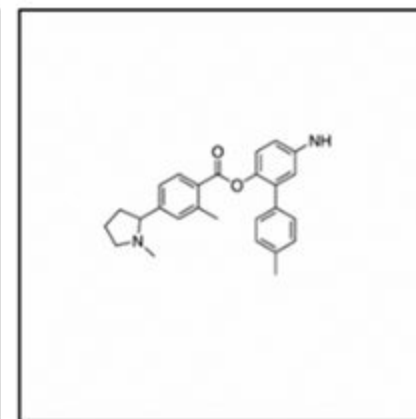
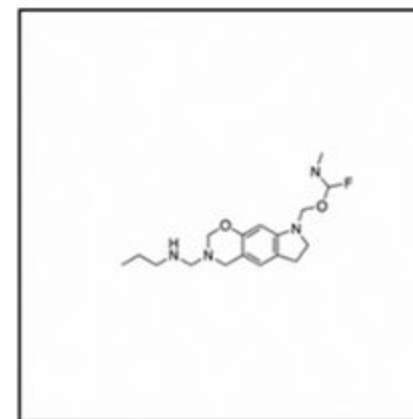
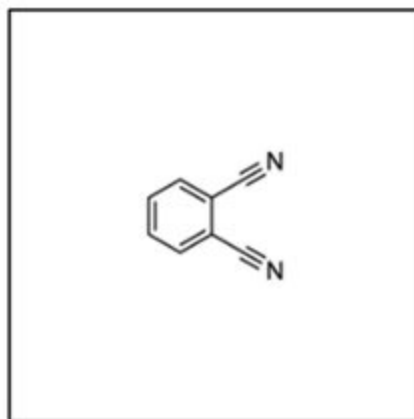
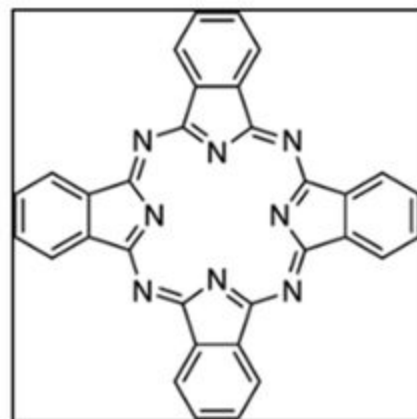
» 프론트 기술 스택

DDPM

- Generative model



- 분자구조 이미지 학습
- 분자구조 이미지 생성

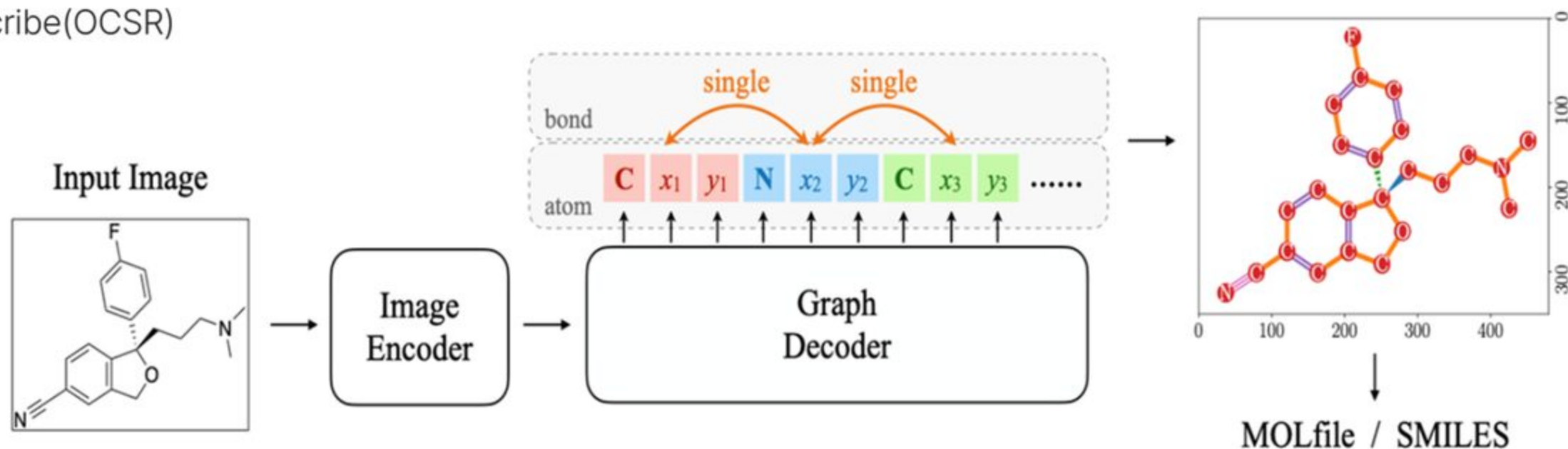


분자구조 이미지 학습 데이터 예시

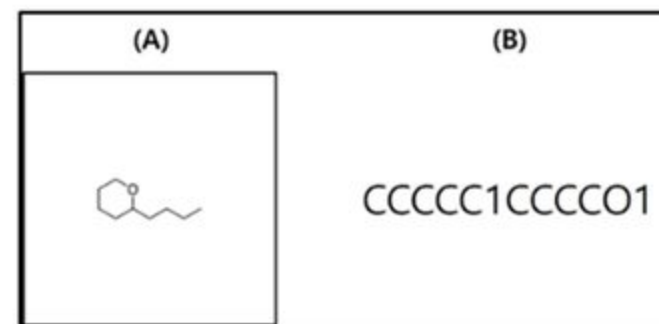
분자구조 이미지 생성 데이터 예시

MolScribe(OCSR)

- MolScribe(OCSR)



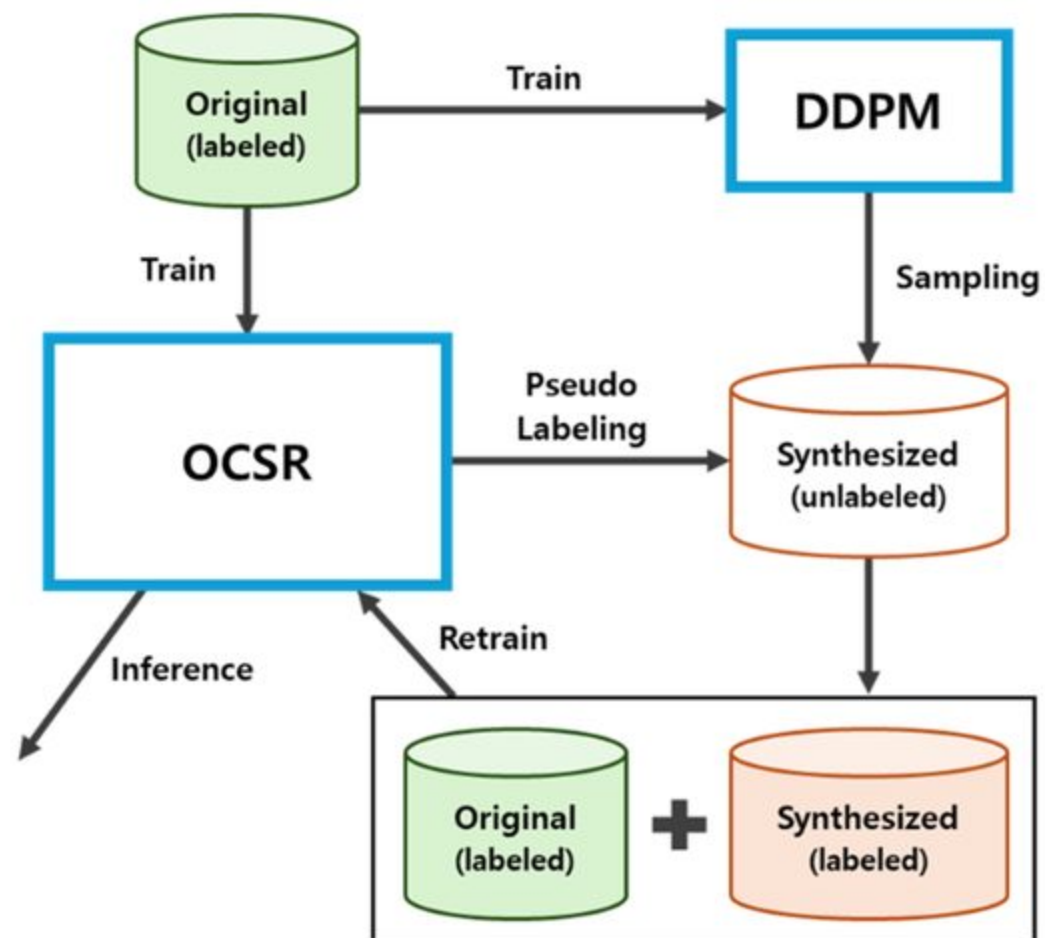
- Optical chemical structure recognition (광학분자구조 인식)
- 분자구조 이미지와 SMILES 쌍으로 학습
- 분자구조 이미지 → SMILES
- * SMILES : 컴퓨터가 화학식을 인식하는 문자열



- MolScribe 학습 데이터 예시

(A) 분자 구조 이미지
(B) SMILES

학습 알고리즘



• DDPM train

분자구조 이미지 학습

• DDPM sampling

분자구조 이미지 생성

• OCSR inference

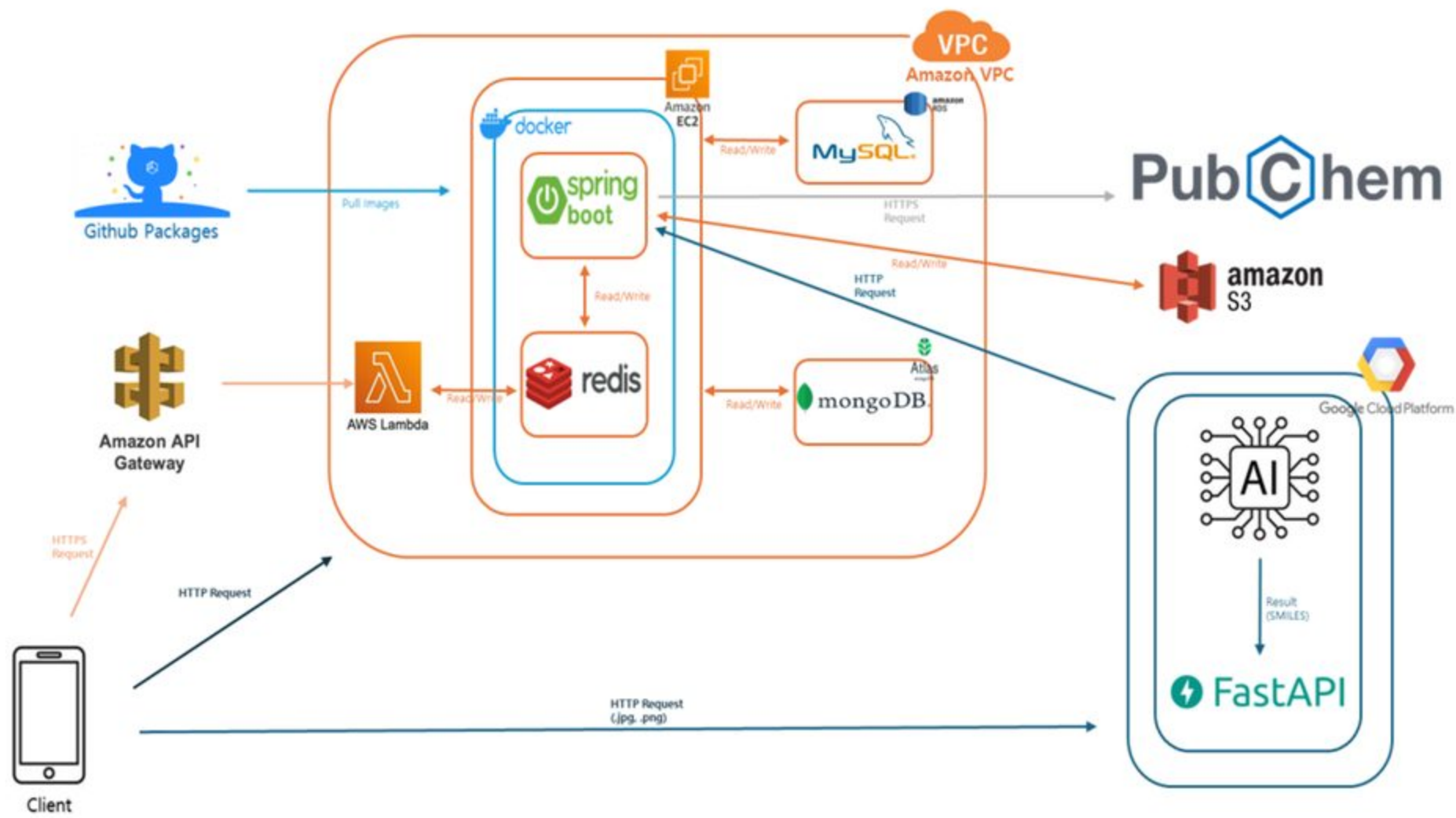
생성된 이미지를 예측해 pseudo labeling 수행

• OCSR retrain

기존 데이터 + pseudo labelling된 synthesized data
Pretrain된 체크포인트를 불러와서 retrain

05

백엔드



서버 통신 설계

05

Spring Boot - DB

```
public class SearchLog extends BaseTime {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id; // SearchLog PK

    @ManyToOne(fetch = FetchType.LAZY) // 지연 로딩으로 설정 (Select 시 Users와 JOIN 방지)
    @JoinColumn(name = "users_id")
    private Users user; // Users 객체 참조

    @Column
    private String log; // 검색어
}
```

```
@Table(indexes = @Index(name = "chemInfoId_index", columnList = "chemInfoId")) // chemInfoId에 대한 Index 설정
public class InfoBookmark extends BaseTime {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id; // Infofav Table PK

    @ManyToOne(fetch = FetchType.LAZY) // 지연 로딩으로 설정 (Select 시 Users와 부분별 JOIN 방지)
    @JoinColumn(name = "user_id")
    private Users user; // Users 객체 참조

    @Column
    private String chemInfoId; // MongoDB의 ChemInfo Document _id 참조

    @Column
    private String molecularFormula; // 분자식
}
```

```
public class ChemInfo {

    @Id
    private String id; // ChemInfo PK

    private Long cid; // 분자 고유 ID

    private String inchiName; // 국제 표준화된 이름

    private String molecularFormula; // 분자식

    private Double molecularWeight; // 분자량 (g/mol)

    private String canonicalSmiles; // 표준 SMILES

    @Indexed(name = "index_smiles", unique = true) // Unique Index 설정
    private String isomericSmiles; // 이성질체 SMILES (이성질체 + 분자식은 같지만 분자 구조가 다른 분자)

    private String inchi; // 국제 화학 식별자

    private String inchiKey; // InChI Key 값

    private String description; // 화합물에 대한 설명

    private String image2DUrl; // 2D 이미지 경로

    private List<String> synonyms; // 상위 3개의 synonyms 값
}
```

DB 설계

회원정보

엄격한 관리가 필요

RDBMS - MySQL

분자정보

비정형 데이터

가장 많이 사용하는 데이터

NoSQL - MongoDB



자주 사용하는 Column(Field)의 경우

인덱스를 설정해 탐색 속도 향상

05

Spring Boot - DB

```
public class Synonyms {  
  
    @Id  
    private String id;    // Synonyms PK  
  
    @DBRef // Reference 설정  
    private ChemInfo chemInfo; // ChemInfo 객체 참조  
  
    @TextIndexed // TextIndex 생성 (Full Text Search 이용)  
    private List<String> synonyms; // 비슷한 화합물들  
}
```

```
public class Tokens {  
  
    @Id // Key (Auto Increase)  
    private Long id;    // Primary Key  
  
    private String refreshToken; // Refresh Token 값  
    ⚡  
    @Indexed // 검색이용  
    private String accessToken; // Access Token 값  
  
    @TimeToLive(unit = TimeUnit.MILLISECONDS)  
    private long ttl; // 데이터 만료시간 (Refresh Token 만료 시간과 같음)
```

DB 설계

검색어로 사용되는 Synonyms - Text Index로 설정
MongoDB의 Full Text Search 기능 사용해 검색 성능 향상



Token은 로그인 이후 통신에서 계속 사용
Token의 유효성 검증과 빠르게 읽기 위해 Redis 사용
TTL을 설정해 Refresh Token이 만료되면 자동 삭제

05

Spring Boot - DB

ID	Name	Molecular Formula	Molecular Weight	Canonical SMILES	Isomeric SMILES	InChI	InChI Key	Description	Image2D URL	Synonyms
1	3-acetyloxy-4-(trimethylazaniumyl)butanoate	C9H17NO4	203.24	CC(=O)OC(CC(=O)[O-])C[N+](C)(C)C	CC(=O)OC(CC(=O)[O-])C[N+](C)(C)C	InChI=1S/C9H17NO4/c1-7(11)14-8(5-9(12)13)6-10(2,3)4/h8H,5-6H2,1-4H3	RDHQFKQIGNGIED-UHFFFAOYSA-N	0-acetylcarnitine is an 0-acylcarnitine having acetyl as the acyl subs...	https://canchem-images.s3.ap-northeast-2.amazonaws.com/3c2f07388b4146c...	
2	(2-acetyloxy-3-carboxypropyl)-trimethylazanium	C9H18NO4+	204.24	CC(=O)OC(CC(=O)O)C[N+](C)(C)C	CC(=O)OC(CC(=O)O)C[N+](C)(C)C	InChI=1S/C9H17NO4/c1-7(11)14-8(5-9(12)13)6-10(2,3)4/h8H,5-6H2,1-4H3/p+	RDHQFKQIGNGIED-UHFFFAOYSA-O		https://canchem-images.s3.ap-northeast-2.amazonaws.com/4a075ca6bee9406...	

DB 설계

분자구조 이미지 파일

데이터의 크기가 커 DB에 저장 후 읽는 것이 비효율적



Amazon S3 서비스를 이용해 이미지 저장
저장된 이미지의 URL을 DB에 저장

05

Spring Boot - 보안

보안 설계

Spring Security, JWT 사용

JWT 토큰 검증을 위해 필터를 통신의 앞 단에 위치



토큰이 만료되기 전 갱신해 통신 유지 가능
(단, Refresh Token이 만료되기 전까지)

로그아웃 시 해당 토큰을 Redis에서 삭제
(토큰을 탈취 당해도 서버에 접근 불가)

```
@Configuration
@EnableWebSecurity // Spring Security 적용을 가능하게 함
@ComponentScan(basePackages = {"com.example.springbootsearch.jwt"})
@EnableRedisRepositories
public class SecurityConfig {

    private final JwtProvider jwtProvider;
    private final JwtHandler jwtHandler;
    private final TokenRepository tokenRepository;

    @Bean
    protected SecurityFilterChain configure(HttpSecurity http) throws Exception {
        http
            // JWT로 인증되므로 통신의 앞단에 csrf disable
            .csrf(AbstractHttpConfigurer::disable)
            // 보안 사용 X
            .sessionManagement((sessionManagement) ->
                sessionManagement.sessionCreationPolicy(SessionCreationPolicy.STATELESS)
            )
            // Jwt Handler 등록
            .exceptionHandling(httpSecurityExceptionHandlingConfigurer ->
                httpSecurityExceptionHandlingConfigurer.authenticationEntryPoint(jwtHandler)
                .accessDeniedHandler(jwtHandler)
            )
            // JwtFilter를 Security 객체로 등록
            .addFilterBefore(new JwtFilter(jwtProvider, tokenRepository), UsernamePasswordAuthenticationFilter.class) // JwtFilter를 UsernamePasswordAuthenticationFilter보다 먼저 실행
            // 세션 관리 설정 X
            .authorizeHttpRequests((authorizeHttpRequests) ->
                authorizeHttpRequests.requestMatchers("/api/login/logout", "/api/login/refresh").permitAll() // 로그인 요청은 누구나 가능
                .anyRequest().authenticated() // 그외 다른 요청들은 모든 인증필수
            );

        http.addFilterBefore(new JwtExceptionHandler(), JwtFilter.class); // JwtExceptionHandler를 JwtFilter보다 먼저 실행

        return http.build();
    }
}
```

05

Spring Boot - 보안

보안 설계

OAuth 2.0 프로토콜을 이용해 부족한 보안을 보충

네이버에 로그인 하여 로그인에 대한 보안을 위임

계정 정보를 받아와 자동 회원가입 및 로그인 허용



```

/* 네이버 SDK 객체 초기화 */
val naverClientId = getString(R.string.naver_client_id) // 발급 받은 naver client id 값
val naverClientSecret = getString(R.string.naver_client_secret) // 발급 받은 naver client secret 값
val naverClientName = getString(R.string.naver_client_name) // 이름 이름
NaverIdLoginSDK.initialize(this, naverClientId, naverClientSecret, naverClientName) // 네이버 객체 초기화

googleAuthLauncher() // Google 로그인 초기화
googleSignInClient = getGoogleClient() // Google Client 초기화

/* Google 로그인 버튼 클릭 */
binding.btnGoogleLogin.setOnClickListener {
    resultLauncher.launch(googleSignInClient.signInIntent()) // 구글 로그인 창으로 넘어감

    /* 로그인한 상태 */
    binding.btnNaverLogin.visibility = View.INVISIBLE
    binding.btnGoogleLogin.visibility = View.INVISIBLE
    binding.btnNaverLogout.visibility = View.VISIBLE
}

/* 네이버 로그인 버튼 클릭 */
binding.btnNaverLogin.setOnClickListener {

    /* 네이버 Access Token 받기 */
    val oauthLoginCallback = object : OAuthLoginCallback {
        override fun onSuccess() {
            // 네이버 로그인 인증이 성공했을 때 수행할 코드 추가
            val accessToken = NaverIdLoginSDK.getAccessToken().toString() // 접근 토큰
            val refreshToken = NaverIdLoginSDK.getRefreshToken().toString() // 결산 토큰
            val drefreshToken = NaverIdLoginSDK.getRefreshToken().toString() // 결산 토큰
            val expiresAt = NaverIdLoginSDK.getExpiresAt().toString() // 만료 기한 (초)
            val type = NaverIdLoginSDK.getTokenType().toString() // 토큰 타입
            val state = NaverIdLoginSDK.getState().toString() // 로그인 연스탈트의 현재 상태

            Toast.makeText(this@MainActivity, "Access Token : ${accessToken}\n" +
                "Refresh Token : ${refreshToken}\n" +
                "Expires at : ${expiresAt}\n" +
                "Type : ${type}\n" +
                "State : ${state}", Toast.LENGTH_SHORT).show()

            NidOAuthLogin().callProfileApi(nidProfileCallback)

            /* 로그인한 상태의 View 설정 */
            binding.btnNaverLogin.visibility = View.INVISIBLE
            binding.btnGoogleLogin.visibility = View.INVISIBLE
            binding.btnNaverLogout.visibility = View.VISIBLE
        }
    }

    NidOAuthLogin().callLoginApi(nidLoginCallback)
}

```

05

Spring Boot - 성능 최적화

최적화

하나의 분자정보를 위해 4개의 외부 API가 필요

각각의 API 통신 시간은 0.8~1.6초
(Synchronous Processing시 3.2~5.4초 이상 소요)



해결 방안

WebFlux의 WebClient를 이용
4개의 API를 한번에 Asynchronous Processing

4개의 API 요청시 0.8~1.6초에 처리

```

// 4개 외부 API를 한번에 요청하고 결과를 처리하는 메서드
public CheckInfo requestInfo(Keyword keyword) {
    // 1. 키워드 검색
    Mono<CheckInfo> checkInfo = getCheckInfo(keyword);
    // 2. 동의어 검색
    Mono<SynonymResponse> synonymInfo = getSynonymInfo(keyword);
    // 3. 설명 검색
    Mono<DescriptionResponse> descriptionInfo = getDescription(keyword);
    // 4. 이미지 검색
    Mono<ImageStreamSource> imageInfo = getImage(keyword);
    CheckInfo checkInfo;

    try {
        Tuple4<CheckInfo, SynonymResponse, DescriptionResponse, ImageStreamSource> tuple4 = Flux.zip(checkInfo, synonymInfo, descriptionInfo, imageInfo).block();
        checkInfo = tuple4.get(0);

        // 5. 키워드 검색 결과 처리
        CheckInfo info = checkInfoRepository.findByKeyword(checkInfo.getKeyword()).orElse(null);
        if (info != null) {
            return info;
        }

        // 6. 키워드 검색 결과 처리
        byte[] bytes = tuple4.get(3).getInputStream().readAllBytes();
        try {
            String imageUrl = new String(bytes, "UTF-8");
            checkInfo.setImageUrl(imageUrl);
        } catch (IOException e) {
            log.error("IO Error", e);
        }

        // 7. 동의어 검색 결과 처리
        if (tuple4.get(1).getSynonyms().size() > 0) {
            checkInfo.setSynonyms(tuple4.get(1).getSynonyms());
        }

        // 8. 설명 검색 결과 처리
        checkInfo.setDescription(tuple4.get(2).getDescription());

        log.debug("Save URL: {}, timestamp: {}, checkInfo: {}", checkInfo.getImageUrl(), LocalDateTime.now());
    } catch (WebClientRequestException e) {
        log.error("WebClient 요청 실패: {}, timestamp: {}, e: {}", e, LocalDateTime.now());
        throw new CustomException(ErrorCode.EXTERNAL_API_REQUEST_FAILED);
    } catch (CustomException e) {
        log.error("기타 예외 발생: {}, timestamp: {}, e: {}", e, LocalDateTime.now());
        throw new CustomException(ErrorCode.EXTERNAL_API_REQUEST_FAILED);
    } catch (Exception e) {
        log.error("API 요청 실패: {}, timestamp: {}, e: {}", e, LocalDateTime.now());
        throw new CustomException(ErrorCode.EXTERNAL_API_REQUEST_FAILED);
    }

    return checkInfo;
}

```

05

Spring Boot - 성능 최적화

```
@Modifying
@Query("delete from SearchLog s where s.id in :searchLogList") // 여러 데이터를 한 번에 삭제 (성능 향상)
void deleteAllByList(@Param("searchLogList") List<Long> searchLogList); // 선택된 검색 기록 목록 삭제

@Modifying
@Query("delete from SearchLog s where s.user in :user") // 여러 데이터를 한 번에 삭제 (성능 향상)
void deleteAllByUser(@Param("user") Users user); // 해당 유저 대한 모든 기록 삭제

@Modifying
@Query("delete from SearchLog s where s.user.id in :users_id") // 여러 데이터를 한 번에 삭제 (성능 향상)
void deleteAllByUser_Id(@Param("users_id") Long users_id); // 해당 유저 대한 모든 기록 삭제
```



```
@Modifying
@Query("delete from InfoBookmark b where b.id in :bookmarkList") // 여러 데이터를 한 번에 삭제 (성능 향상)
void deleteAllByList(@Param("bookmarkList") List<Long> bookmarkList); // 선택된 즐겨찾기 목록 삭제

@Modifying
@Query("delete from InfoBookmark b where b.user in :user") // 여러 데이터를 한 번에 삭제 (성능 향상)
void deleteAllByUser(@Param("user") Users user); // 해당 유저에 대한 정보 전부 삭제

@Modifying
@Query("delete from InfoBookmark b where b.user.id in :users_id") // 여러 데이터를 한 번에 삭제 (성능 향상)
void deleteAllByUser_Id(@Param("users_id") Long users_id); // 해당 유저에 대한 정보 전부 삭제
```

최적화

Spring Data JPA

여러 개의 데이터 삭제 시 하나씩 삭제 요청을 보냄
(DB 성능 저하)

JPQL을 이용해 한 번의 요청으로 여러 개의 데이터를 삭제

05

Spring Boot - 성능 최적화

최적화

User (부모 테이블)

SearchLog 및 InfoBookmark (자식 테이블)

검색기록 및 즐겨찾기 조회 시

회원 정보도 join 연산을 통해 같이 제공



```
@ManyToOne(fetch = FetchType.LAZY) // 지연 로딩으로 설정 (Select 시 Users와 JOIN 방지)
@JoinColumn(name = "users_id")
private Users user; // Users 객체 참조
```

검색기록이나 즐겨찾기 정보를 조회할 때 회원정보 불필요

지연 로딩을 통해 join 연산을 하지 않고 저장된 값을 이용
DB의 불필요한 연산을 줄여 통신 시간 감소

05

Spring Boot - 성능 최적화

최적화

JWT 토큰의 갱신을 위해 서버에 반복적인 요청이 필요

토큰의 갱신 부분을 분리 시켜 서버의 부하 감소



Spring Boot 기반으로 토큰 갱신 기능을 AWS Lambda로 구현

Serverless 형태로 Amazon API Gateway 서비스로 배포

기존의 서버의 부하를 감소

함수 개요 정보

다이어그램

템플릿



+ 대상 추가

API 게이트웨이

+ 트리거 추가

05

CanChem ERD

이미지 파일(AWS S3)			Images
	2D 이미지 파일		png

분자구조 이미지

토큰정보(Redis)			Tokens
Key	id		String
Access Token	access Token		String
Refresh Token	refresh Token		String

로그인 정보

분자정보(MongoDB)			Cheminfo
ID	_id		string
분자고유ID	cid		string
유기화합물이름	inpacName		string
화학식	molecularFormula		string
분자량	molecularWeight		double
이성질체SMILES	isomericSmiles		string
국제화학식별자	inchi		string
InChiKey값	inchiKey		string
표준SMILES	canonicalSmiles		string
비슷한화합물들	synonyms		string
설명	description		string
2D이미지URL	image2DUrl		string

동의어(MongoDB)			synonyms
PK	id		png
분자정보	cheminfo_id		svg
동의어	synonyms		List<string>

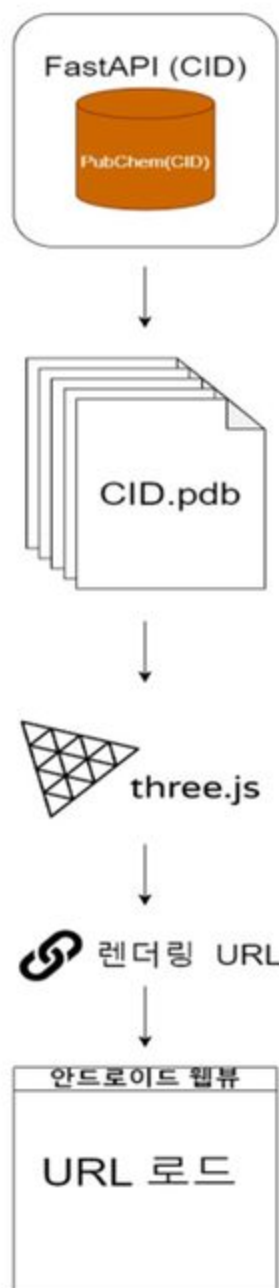
회원정보(MySQL)			Users
	PK	id	bigint
	식별정보	userid	varchar(50)
	이메일	email	varchar(40)
	유저이름	name	varchar(20)
	닉네임	nickname	varchar(20)
	성별	gender	varchar(5)
	휴대전화번호	mobile	varchar(20)
	프로필사진	profileImage	varchar(100)
	권한	roleType	int
	가입날짜	createdDate	datetime(6)
	수정날짜	modifiedDate	datetime(6)
	가입경로	signUpType	int

검색 리스트(MySQL)			Search
	ID	id	bigint
	회원정보 ID	users_id	bigint
	검색어	text	varchar(100)
	생성된날짜	createdDate	datetime(6)

정보 즐겨찾기 리스트(MySQL)			Search
	ID	id	bigint
	회원정보ID	users_id	bigint
	분자정보ID	text	varchar(100)
	화학식	createdDate	datetime(6)
	생성된날짜	createdDate	datetime(6)

06

안드로이드 3D 분자구조 렌더링



안드로이드

검색한 분자구조의 CID값을 FastAPI로 전송

FastAPI

RDKit 라이브러리를 사용
CID값에 해당하는 PDB파일 생성

PDB파일을 HTML 페이지로 전달해 Three.js로 렌더링

렌더링된 HTML 페이지의 URL을 생성하여 반환

안드로이드

반환된 URL으로 웹뷰를 사용해 화면에 표시

06

CanChem 스토리보드

로그인 화면



네이버 연동 로그인

홈 화면



06

CanChem 스토리보드

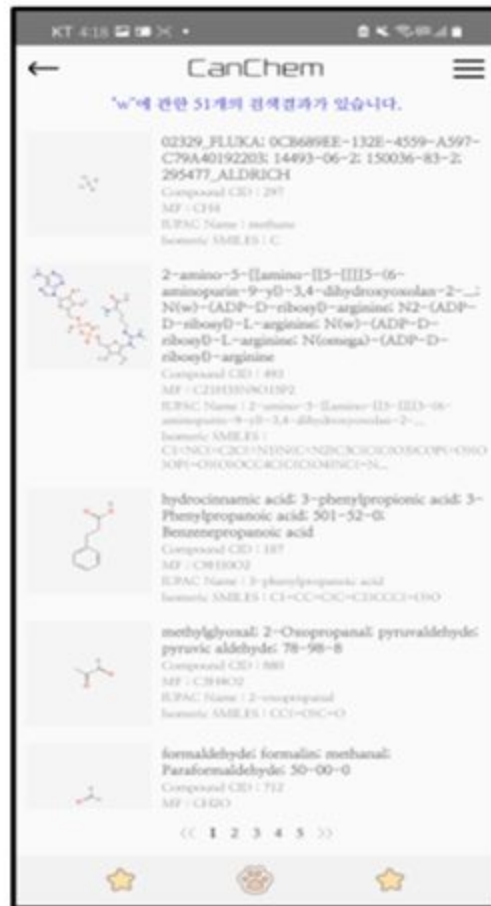
홈 화면



분자 정보 검색

1. text
2. 카메라, 갤러리
(분자 구조 사진)

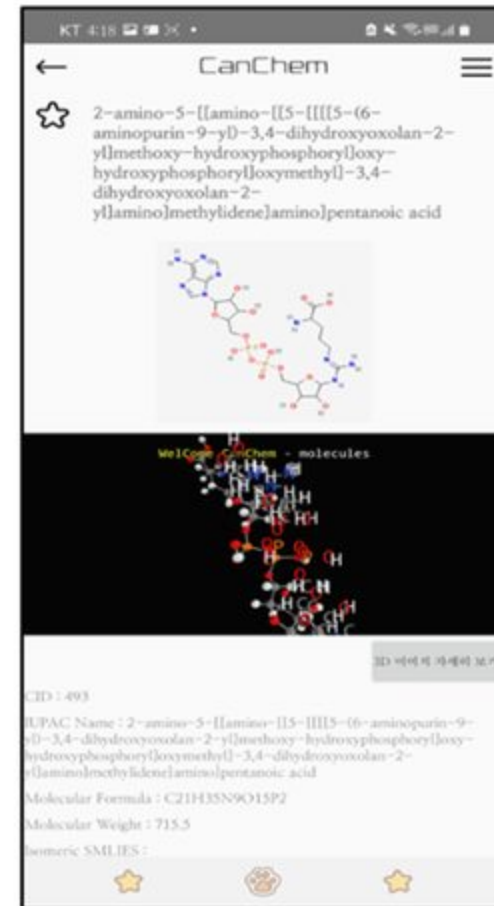
검색 결과 페이지



원하는 정보 터치

검색결과가 하나일 때
바로 분자정보로 이동

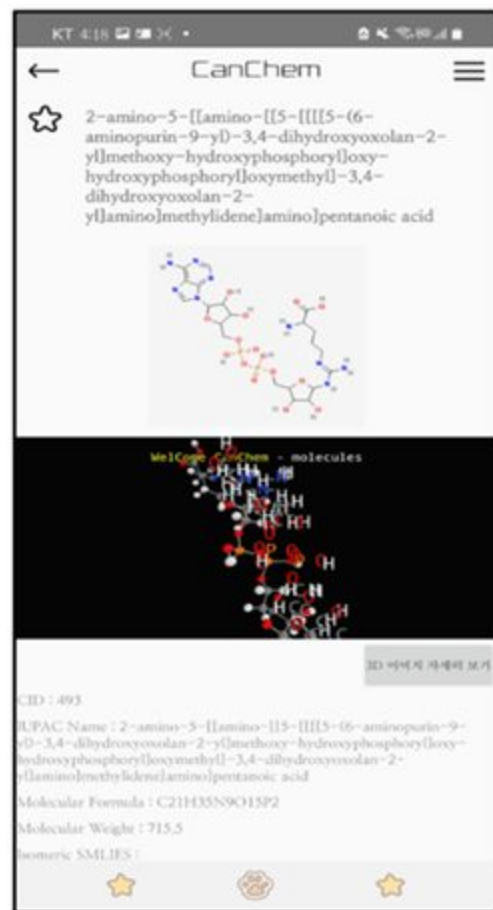
분자정보 페이지



06

CanChem 스토리보드

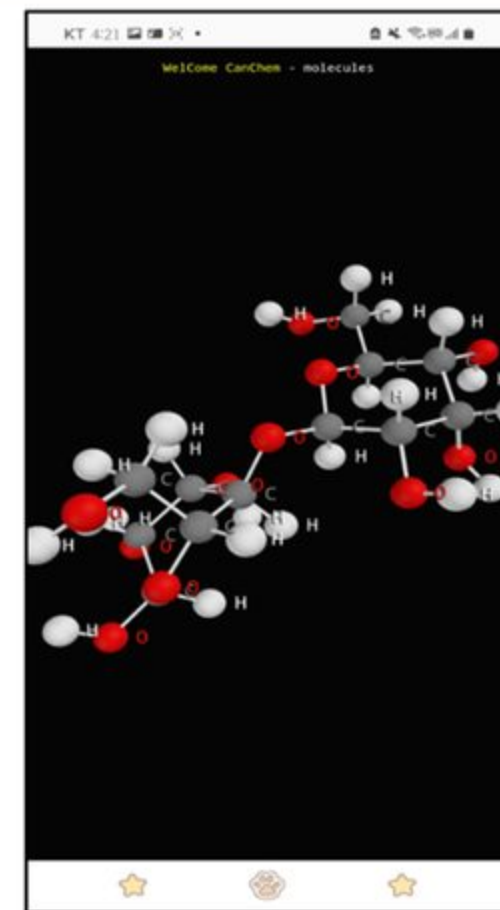
분자정보 페이지



3D 분자구조 이미지

- 줌인, 줌아웃
- 스크롤로 시야각 변경

3D 분자구조 이미지



06

CanChem 스토리보드

홈 화면



메뉴 바 터치 시
마이페이지
및
로그아웃, 회원탈퇴

마이페이지



06

CanChem 스토리보드

마이페이지 세부 목록

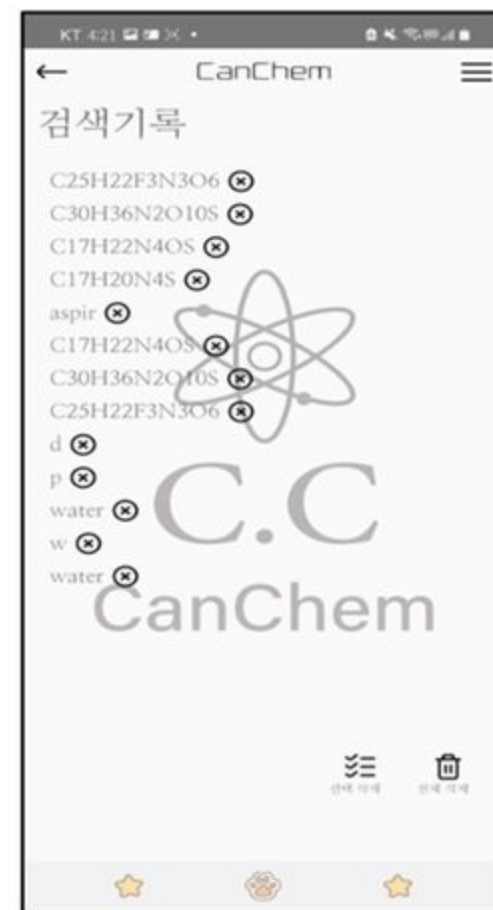
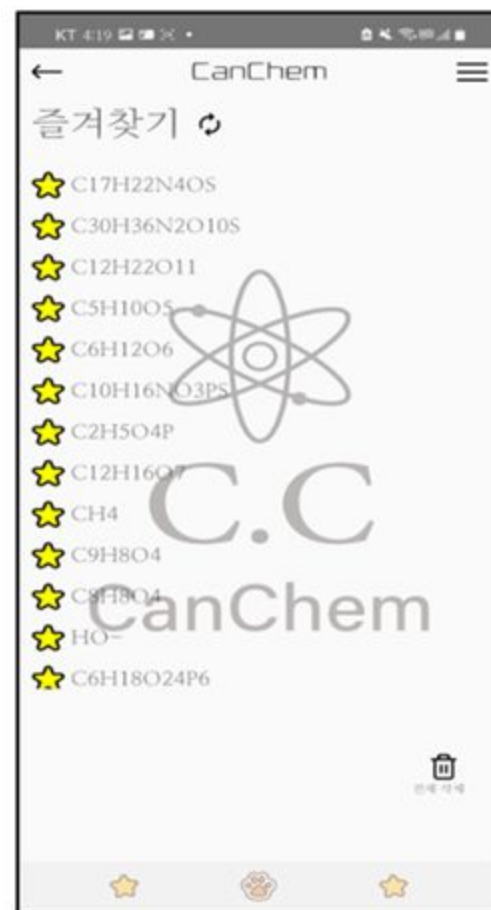


즐거찾기
검색기록
터치 시
목록 페이지로 이동



홈 버튼 터치시
홈화면으로 이동

즐거 찾기 목록 / 검색기록 목록



목록 편집 기능

- 단일 삭제
- 선택 삭제
- 전체 삭제

소프트웨어캡스톤디자인[02]

Q&A

감사합니다

김진혁, 박홍주, 송태웅, 이동민, 이연제